

```
$$$ src.controle.projeto
```

```
from util.gerais import imprimir_objetos, imprimir_objeto, imprimir_objetos_internos,
imprimir_objetos_associacao_filtros
from util.data import Data
from entidades.seguradora import inserir_seguradora, Seguradora, get_seguradoras
from entidades.pecas import Peca, PeçaMecânica, PeçaLatária
from entidades.sinistro import inserir_sinistro, Sinistro, get_sinistros
from entidades.orcamento import criar_orcamento, get_orcamentos, selecionar_orcamento
```

```
def cadastrar_seguradoras():
    inserir_seguradora(Seguradora(nome='Porto Seguro', cidade='Dourados',
cobertura_percentual=90))
    inserir_seguradora(Seguradora(nome='Bradesco Seguros', cidade='Campo Grande',
cobertura_percentual=80))
    inserir_seguradora(Seguradora(nome='SulAmérica', cidade='Ponta porã',
cobertura_percentual=70))
    inserir_seguradora(Seguradora(nome='Mapfre', cidade='Dourados', cobertura_percentual=85))
    inserir_seguradora(Seguradora(nome='Allianz', cidade='Campo Grande',
cobertura_percentual=75))
    inserir_seguradora(Seguradora(nome='Tokio Marine', cidade='Ponta Porã',
cobertura_percentual=90))
    inserir_seguradora(Seguradora(nome='Azos Seguros', cidade='São Paulo',
cobertura_percentual=99))
    inserir_seguradora(Seguradora(nome='Liberty Seguros', cidade='Dourados',
cobertura_percentual=55))
    inserir_seguradora(Seguradora(nome='Sompo Seguros', cidade='Maracaju',
cobertura_percentual=75))
```

```
def cadastrar_sinistros():
    sinistro = Sinistro(numero='1', cliente='Leon', telefone='67 99888-1100')
    inserir_sinistro(sinistro)
    sinistro.inserir_pecas(PeçaMecânica(código=11, nome='bandeja', categoria='OEM', preco=245,
tipo='suspensão', prazo_garantia='90 dias', mão_obra_própria=False))
    sinistro.inserir_pecas(PeçaMecânica(código=12, nome='pivô', categoria='OEM', preco=250,
tipo='suspensão', prazo_garantia='90 dias', mão_obra_própria=False))

    sinistro = Sinistro(numero='6', cliente='Scott', telefone='67 98988-1101')
    inserir_sinistro(sinistro)
    sinistro.inserir_pecas(PeçaLatária(código=21, nome='para-choque', categoria='Original',
preco=200, tipo='externo', cor='preto', mão_obra_própria=False))
    sinistro.inserir_pecas(PeçaLatária(código=22, nome='capô', categoria='Original', preco=180,
tipo='externo', cor='cinza', mão_obra_própria=False))

    sinistro = Sinistro(numero='11', cliente='Kennedy', telefone='67 96898-1000')
    inserir_sinistro(sinistro)
    sinistro.inserir_pecas(PeçaMecânica(código=31, nome='amortecedor', categoria='Original',
preco=180, tipo='suspensão', prazo_garantia='90 dias', mão_obra_própria=True))
    sinistro.inserir_pecas(PeçaMecânica(código=33, nome='kit batente', categoria='Original',
preco=220, tipo='suspensão', prazo_garantia='180 dias', mão_obra_própria=True))
```

```

sinistro = Sinistro(numero='19', cliente='Chris', telefone='44 90898-0100')
inserir_sinistro(sinistro)
sinistro.inserir_pecas(PecaLataria(codigo=41, nome='painel central', categoria='OEM', preco=135,
tipo='interno', cor='cinza', mão_obra_própria=True))
sinistro.inserir_pecas(PecaLataria(codigo=44, nome='volante', categoria='Original', preco=60,
tipo='interno', cor='cinza', mão_obra_própria=True))

sinistro = Sinistro(numero='98', cliente='Redfield', telefone='66 91681-1198')
inserir_sinistro(sinistro)
sinistro.inserir_pecas(PecaMecânica(codigo=51, nome='pistão', categoria='Genuína', preco=150,
tipo='motor', prazo_garantia='365 dias', mão_obra_própria=True))
sinistro.inserir_pecas(PecaMecânica(codigo=55, nome='biela', categoria='Genuína', preco=230,
tipo='motor', prazo_garantia='365 dias', mão_obra_própria=True))

sinistro = Sinistro(numero='102', cliente='Jill', telefone='67 99234-8811')
inserir_sinistro(sinistro)
sinistro.inserir_pecas(PecaMecânica(codigo=61, nome='amortecedor', categoria='OEM',
preco=150, tipo='suspensão', prazo_garantia='180 dias', mão_obra_própria=False))
sinistro.inserir_pecas(PecaMecânica(codigo=62, nome='mola', categoria='OEM', preco=210,
tipo='suspensão', prazo_garantia='180 dias', mão_obra_própria=False))

sinistro = Sinistro(numero='145', cliente='Claire', telefone='67 98112-2290')
inserir_sinistro(sinistro)
sinistro.inserir_pecas(PecaLataria(codigo=71, nome='retrovisor', categoria='Original', preco=210,
tipo='externo', cor='cinza', mão_obra_própria=False))
sinistro.inserir_pecas(PecaLataria(codigo=72, nome='porta dianteira', categoria='Original',
preco=170, tipo='externo', cor='cinza', mão_obra_própria=False))

sinistro = Sinistro(numero='201', cliente='Ada', telefone='65 99677-4400')
inserir_sinistro(sinistro)
sinistro.inserir_pecas(PecaMecânica(codigo=81, nome='radiador', categoria='Genuína',
preco=160, tipo='motor', prazo_garantia='365 dias', mão_obra_própria=True))
sinistro.inserir_pecas(PecaMecânica(codigo=82, nome='bieleta', categoria='Genuína', preco=140,
tipo='suspensão', prazo_garantia='365 dias', mão_obra_própria=True))

sinistro = Sinistro(numero='250', cliente='Rebecca', telefone='67 99123-4455')
inserir_sinistro(sinistro)
sinistro.inserir_pecas(PecaLataria(codigo=91, nome='farol', categoria='Original', preco=140,
tipo='externo', cor='prata', mão_obra_própria=False))
sinistro.inserir_pecas(PecaLataria(codigo=92, nome='lanterna', categoria='OEM', preco=290,
tipo='externo', cor='preto', mão_obra_própria=True))

def cadastrar_orcamentos():
    criar_orcamento(numero_sinistro='1', nome_seguradora='Porto Seguro', data=Data(dia=12,
mês=6, ano=2026))
    criar_orcamento(numero_sinistro='6', nome_seguradora='Bradesco Seguros', data=Data(dia=10,
mês=10, ano=2026))
    criar_orcamento(numero_sinistro='11', nome_seguradora='SulAmérica', data=Data(dia=3,
mês=8, ano=2026))
    criar_orcamento(numero_sinistro='19', nome_seguradora='Mapfre', data=Data(dia=2, mês=5,
ano=2026))

```

```

    criar_orcamento(numero_sinistro='98', nome_seguradora='Allianz', data=Data(dia=28, mês=4,
ano=2026))
    criar_orcamento(numero_sinistro='102', nome_seguradora='Tokio Marine', data=Data(dia=15,
mês=7, ano=2026))
    criar_orcamento(numero_sinistro='145', nome_seguradora='Azos Seguros', data=Data(dia=22,
mês=9, ano=2026))
    criar_orcamento(numero_sinistro='201', nome_seguradora='Liberty Seguros', data=Data(dia=5,
mês=11, ano=2026))
    criar_orcamento(numero_sinistro='250', nome_seguradora='Sompo Seguros', data=Data(dia=18,
mês=12, ano=2026))

if __name__ == '__main__':
    cadastrar_seguradoras()
    imprimir_objetos(cabeçalho= '\n Seguradoras: Nome - Cidade - Cobertura percentual',
objetos=get_seguradoras().values())

    cadastrar_sinistros()
    imprimir_objetos(cabeçalho='\n Sinistros: Número - Cliente - Telefone',
        objetos=get_sinistros().values())

    print('\n Sinistros: Número - Cliente - Telefone')
    print(' - Peças: Nome - Categoria - Preço - Tipo - Prazo de Garantia - Mão de obra própria')
    for índice, sinistro in enumerate(get_sinistros().values()):
        imprimir_objeto(índice=índice, objeto_str=str(sinistro))
        imprimir_objetos_internos(sinistro.pecas.values())

    cadastrar_orcamentos()
    cabecalho_orcamento = ('Orcamento: Numero do Sinistro - Nome da Seguradora - Data do
orcamento')
    cabecalho_orcamento_filtros = (cabecalho_orcamento + '\n -- Cobertura Percentual - Telefone no
Sinistro - Precos das pecas'
        + '\n PeçaMecânica[Tipo], PeçaLataria[Cor]')

    imprimir_objetos('\n' + cabecalho_orcamento, get_orcamentos())

    filtros, orcamentos_selecionados = selecionar_orcamento()
    imprimir_objetos_associação_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,
filtros)

    filtros, orcamentos_selecionados = selecionar_orcamento(data_mínima_orcamento=Data(dia=1,
mês=5, ano=2026))
    imprimir_objetos_associação_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,
filtros)

    filtros, orcamentos_selecionados = selecionar_orcamento(Data(1, 5, 2026),
valor_máximo_peca=255)
    imprimir_objetos_associação_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,
filtros)

    filtros, orcamentos_selecionados = selecionar_orcamento(Data(1, 5, 2026), 255,
tipo_peça_mecânica='suspensão')
    imprimir_objetos_associação_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,

```

filtros)

```
    filtros, orcamentos_selecionados = selecionar_orcamento(Data(1, 5, 2026), 255, 'suspensão',  
cor_peça_lataria='cinza')
```

```
    imprimir_objetos_associacao_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,  
filtros)
```

```
    filtros, orcamentos_selecionados = selecionar_orcamento(Data(1, 5, 2026), 255, 'suspensão',  
'cinza', cobertura_mínima_seguradora=85)
```

```
    imprimir_objetos_associacao_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,  
filtros)
```

```
    filtros, orcamentos_selecionados = selecionar_orcamento(Data(1, 5, 2026), 255, 'suspensão',  
'cinza', 85, prefixo_telefone_cliente=67)
```

```
    imprimir_objetos_associacao_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,  
filtros)
```

```
$$$ src.entidades.seguradora
```

```
seguradoras = {}
```

```
def get_seguradoras(): return seguradoras
```

```
def inserir_seguradora(seguradora):
```

```
    nome_seguradora = seguradora.nome
```

```
    if nome_seguradora not in seguradoras.keys():
```

```
        seguradoras[nome_seguradora] = seguradora
```

```
    return True
```

```
    else:
```

```
        print('Seguradora ' + nome_seguradora + ' tem cadastro')
```

```
    return False
```

```
class Seguradora:
```

```
    def __init__(self, nome, cidade, cobertura_percentual):
```

```
        self.nome = nome
```

```
        self.cidade = cidade
```

```
        self.cobertura_percentual = cobertura_percentual
```

```
    def __str__(self):
```

```
        formato = '{ } {:<18} { } {:<14} { } {:<3} { }'
```

```
        seguradora_formatada = formato.format('|', self.nome, '|', self.cidade, '|',
```

```
str(self.cobertura_percentual), '|')
```

```
        return seguradora_formatada
```

\$\$\$ src.entidades.pecas

class Peca:

```
def __init__(self, código, nome, categoria, preco, mão_obra_própria):
    self.código = código
    self.nome = nome
    self.categoria = categoria if categoria in ('Original', 'Genuína', 'OEM') else 'indefinida'
    self.preco = preco
    self.mão_obra_própria = mão_obra_própria
```

```
def __str__(self):
    if self.mão_obra_própria: mão_obra_própria_str = ' Mão de obra Própria |'
    else: mão_obra_própria_str = ""
    formato = ' {} {:<15} {} {:<12} {} {:<6} {} {}'
    peca_formatado = formato.format('|', self.nome, '|', self.categoria, '|', f'R$ {self.preco:.2f}', '|',
mão_obra_própria_str)
    return peca_formatado
```

class PeçaMecânica(Peca):

```
def __init__(self, código, nome, categoria, preco, tipo, prazo_garantia, mão_obra_própria):
    super().__init__(código, nome, categoria, preco, mão_obra_própria)
    self.tipo = tipo if tipo in ('suspensão', 'direção', 'motor') else 'indefinida'
    self.prazo_garantia = prazo_garantia
```

```
def __str__(self):
    if self.mão_obra_própria: mão_obra_própria_str = ' Mecânico Próprio |'
    else: mão_obra_própria_str = ""
    formato = ' {} {:<15} {} {:<8} {} {:<12} {} {:<8} {} {:>8} {} {:>20}'
    peca_formatado = formato.format('|', self.nome, '|', self.categoria, '|', f'R$ {self.preco:.2f}', '|',
self.tipo, '|', self.prazo_garantia, '|', mão_obra_própria_str)
    return peca_formatado
```

class PeçaLataria(Peca):

```
def __init__(self, código, nome, categoria, preco, tipo, cor, mão_obra_própria):
    super().__init__(código, nome, categoria, preco, mão_obra_própria)
    self.tipo = tipo if tipo in ('externo', 'interno') else 'indefinida'
    self.cor = cor
```

```
def __str__(self):
    if self.mão_obra_própria: mão_obra_própria_str = ' Funileiro Próprio |'
    else: mão_obra_própria_str = ""
    formato = ' {} {:<15} {} {:<8} {} {:<12} {} {:<8} {} {:>8} {} {:>20}'
    peca_formatado = formato.format('|', self.nome, '|', self.categoria, '|', f'R$ {self.preco:.2f}', '|',
self.tipo, '|', self.cor, '|', mão_obra_própria_str)
    return peca_formatado
```

```

$$$ src.entidades.orcamento

from entidades.seguradora import get_seguradoras
from entidades.sinistro import get_sinistros
from entidades.peca import Peca, PeçaMecânica, PeçaLataria

orcamentos = []

def get_orcamentos(): return orcamentos

def inserir_orcamento(orcamento):
    if orcamento not in orcamentos: orcamentos.append(orcamento)
    else: print ('Orcamento de Pecas tem cadastro --- ' + str(orcamento))

def criar_orcamento(numero_sinistro, nome_seguradora, data):
    sinistro = get_sinistros().get(numero_sinistro)
    if sinistro is None:
        print('Sinistro ' + numero_sinistro + ' não cadastrado')
        return

    seguradora = get_seguradoras().get(nome_seguradora)
    if seguradora is None:
        print('Seguradora ' + nome_seguradora + ' não cadastrada')
        return

    orcamento = Orcamento(sinistro, seguradora, data)
    inserir_orcamento(orcamento)

def selecionar_orcamento(data_mínima_orcamento=None, valor_máximo_peca=None,
    tipo_peça_mecânica=None,
        cor_peça_lataria=None, cobertura_mínima_seguradora=None,
    prefixo_telefone_cliente=None):
    filtros = '\nFiltros -- '
    if data_mínima_orcamento is not None: filtros += 'Data mínima do orcamento: ' +
    str(data_mínima_orcamento)
    if valor_máximo_peca is not None: filtros += ' - Maior valor da peca: ' + str(valor_máximo_peca)
    if tipo_peça_mecânica is not None: filtros += ' - Tipo de peça: ' + str(tipo_peça_mecânica)
    if cor_peça_lataria is not None: filtros += ' - Cor da peça: ' + str(cor_peça_lataria)
    if cobertura_mínima_seguradora is not None: filtros += '\n - Cobertura mínima da seguradora: ' +
    str(cobertura_mínima_seguradora)
    if prefixo_telefone_cliente is not None: filtros += (' - DDD telefone cliente: ' +
    str(prefixo_telefone_cliente))

    orcamentos_selecionados = []
    for orcamento in orcamentos:
        if data_mínima_orcamento is not None and orcamento.data < data_mínima_orcamento:
            continue

        excluir_orcamento = False
        for peca in orcamento.sinistro.pecas.values():
            if valor_máximo_peca is not None and peca.preco > valor_máximo_peca:
                excluir_orcamento = True

```

```

        break

    if isinstance(peca, PeçaMecânica):
        if tipo_peça_mecânica is not None and peca.tipo is not tipo_peça_mecânica:
            excluir_orcamento = True
            break
    elif isinstance(peca, PeçaLataria):
        if cor_peça_lataria is not None and peca.cor is not cor_peça_lataria:
            excluir_orcamento = True
            break
    if excluir_orcamento:
        continue

    if cobertura_mínima_seguradora is not None and orcamento.seguradora.cobertura_percentual
    < cobertura_mínima_seguradora:
        continue

    if prefixo_telefone_cliente is not None and not
    orcamento.sinistro.telefone.startswith(str(prefixo_telefone_cliente)):
        continue

    orcamentos_selecionados.append(orcamento)
    return filtros, orcamentos_selecionados

class Orcamento:
    def __init__(self, sinistro, seguradora, data):
        self.sinistro = sinistro
        self.seguradora = seguradora
        self.data = data

    def __str__(self):
        formato = '{} {:<3} {} {:<19} {} {:<10} {}'
        mostra_formato = formato.format('|', self.sinistro.numero, '|', self.seguradora.nome, '|',
        str(self.data), '|')
        return mostra_formato

    def str_atributos_pecas(self):
        atributos_pecas_str = ""
        for indice, peca in enumerate(self.sinistro.pecas.values()):
            if indice > 0:
                atributos_pecas_str += ' - '
            atributos_pecas_str += f'R$ {peca.preco:.2f} '
            if isinstance(peca, PeçaMecânica):
                atributos_pecas_str += peca.tipo
            elif isinstance(peca, PeçaLataria):
                atributos_pecas_str += peca.cor
        return atributos_pecas_str

    def str_filtro(self):
        formato = '{:>2} {} {:<11} {} {:<43} {}'
        filtro_formatado = formato.format(
            str(self.seguradora.cobertura_percentual) + '%',

```



```
        '|', self.sinistro.telefone, '|', self.str_atributos_pecas(), '|'  
    )  
    return self.__str__() + filtro_formatado
```

```
$$$ src.entidades.sinistro
```

```
sinistros = {}
```

```
def get_sinistros (): return sinistros
```

```
def inserir_sinistro(sinistro):
```

```
    numero_sisnitro = sinistro.numero
```

```
    if numero_sisnitro not in sinistros.keys():
```

```
        sinistros[numero_sisnitro] = sinistro
```

```
    return True
```

```
    else:
```

```
        print('Sinistro ' + numero_sisnitro + ' já tem cadastro')
```

```
    return False
```

```
class Sinistro:
```

```
    def __init__(self, numero, cliente, telefone):
```

```
        self.numero = numero
```

```
        self.cliente = cliente
```

```
        self.telefone = telefone
```

```
        self.pecas = {}
```

```
    def __str__(self):
```

```
        formato = '{} {:<3} {} {:<15} {} {:<14} {}'
```

```
        sinistro_formatado = formato.format('|', self.numero, '|', self.cliente, '|', self.telefone, '|')
```

```
        return sinistro_formatado
```

```
    def inserir_peca(self, peca):
```

```
        nome_peca = peca.nome
```

```
        if nome_peca not in self.pecas.keys(): self.pecas[nome_peca] = peca
```

```
        else: print('Nome ' + nome_peca + ' já foi cadastrada em Sinistro')
```

```
$$$ src.util.data
```

```
from datetime import date
```

```
class Data:
```

```
    def __init__(self, dia, mês, ano):
```

```
        self.dia = dia
```

```
        self.mês = mês
```

```
        self.ano = ano
```

```
    def __str__(self):
```

```
        if self.dia < 10: data_str = '0' + str(self.dia)
```

```
        else: data_str = str(self.dia)
```

```
        if self.mês < 10: data_str += "/0" + str(self.mês) + "/"
```

```
        else: data_str += "/" + str(self.mês) + "/"
```

```
        data_str += str(self.ano)
```

```
        return data_str
```

```
    def __eq__(self, data):
```

```
        if self.dia == data.dia and self.mês == data.mês and self.ano == data.ano: return True
```

```
        return False
```

```
    def __ne__(self, data): return not self == data
```

```
    def __gt__(self, data):
```

```
        if self.ano > data.ano: return True
```

```
        elif self.ano < data.ano: return False
```

```
        if self.mês > data.mês: return True
```

```
        elif self.mês < data.mês: return False
```

```
        if self.dia > data.dia: return True
```

```
        elif self.dia < data.dia: return False
```

```
        return False
```

```
    def __lt__(self, data):
```

```
        if self.ano < data.ano: return True
```

```
        elif self.ano > data.ano: return False
```

```
        if self.mês < data.mês: return True
```

```
        elif self.mês > data.mês: return False
```

```
        if self.dia < data.dia: return True
```

```
        elif self.ano > data.ano: return False
```

```
        return False
```

```
    def __ge__(self, data):
```

```
        if self < data: return False
```

```
        else: return True
```

```
    def __le__(self, data):
```

```
        if self > data: return False
```

```
        else: return True
```

```
    def calcular_idade(self, data_referência=None):
```

```
        if data_referência is None:
```

```
            dia_atual_str, mês_atual_str, ano_atual_str = date.today().strftime("%d/%m/%Y").split('/')
```

```
        dia_referência, mês_referência, ano_referência = int(dia_atual_str), int(mês_atual_str),
int(ano_atual_str)
    else:
        dia_referência, mês_referência, ano_referência = data_referência.dia, data_referência.mês,
data_referência.ano
        idade = ano_referência - self.ano
        if mês_referência < self.mês or (mês_referência == self.mês and dia_referência < self.dia):
idade -= 1
    return idade
```

\$\$\$ src.util.gerais

```
def imprimir_objetos(cabeçalho, objetos, filtros=None):  
    if filtros is not None: print(filtros)  
    print(cabeçalho)  
    for indice, objeto in enumerate(objetos):  
        imprimir_objeto(indice, str(objeto))
```

```
def imprimir_objeto(indice, objeto_str):  
    formato = '{} {} {}'  
    ordem = indice + 1  
    separador = '_'  
    string_formatado = formato.format(f'{ordem:2d}', separador, objeto_str)  
    print(string_formatado)
```

```
def imprimir_objetos_associacao_filtros(cabeçalho, objetos, filtros=None):  
    if filtros is not None: print(filtros)  
    print(cabeçalho)  
    for indice, objeto in enumerate(objetos):  
        imprimir_objeto(indice, objeto.str_filtro())
```

```
def imprimir_objetos_internos (objetos):  
    for objeto in objetos: print(' -' + str(objeto))
```